C 语言模块化编程练习题

> 每题产出一对 `.c` / `.h` 文件，全部**不写** `main`；`.h` 必须有头文件保护宏； 要求**零警告**；代码必须写注释（只写事实性说明：做什么、参数含义、返回值含义、取值范围）。

模块 1　`bitops` —— 位操作工具

文件：`inc/bitops.h` + `src/bitops.c` **对应 PPT**：第 3 讲（位运算 / 掩码）、第 1 讲（`while` / `do…while`）

接口（`inc/bitops.h` 的内容）

```
/* ============================================================================
 * bitops.h — 位操作工具模块
 *
 * 本模块只做计算，不保存状态，返回值就是计算结果本身。
 *
 * 位号约定：pos / hi / lo 都从 0 开始计数，合法范围 0 ~ 31。
 * 位号或位域范围非法（pos > 31；hi < lo；hi > 31）时的返回值：
 *   - 返回 word 的函数（bit_set / bit_clear / bit_toggle / bit_field_set）：返回
word 原值
 *   - 取值的函数（bit_get / bit_field_get / bit_mask_make）：返回 0
 * ============================================================================ */
#ifndef BITOPS_H
#define BITOPS_H

#include <stdint.h>

/* 把 word 的第 pos 位置 1，返回结果。 */
uint32_t bit_set(uint32_t word, uint8_t pos);

/* 把 word 的第 pos 位清 0，返回结果。 */
uint32_t bit_clear(uint32_t word, uint8_t pos);

/* 把 word 的第 pos 位取反，返回结果。 */
uint32_t bit_toggle(uint32_t word, uint8_t pos);

/* 读出 word 的第 pos 位，返回 0 或 1。 */
uint8_t  bit_get(uint32_t word, uint8_t pos);

/* 用 while 循环统计 word 里 1 的个数。
 * 例：0x0F0F0F0F 的结果是 16。 */
uint8_t  bit_count(uint32_t word);

/* 用 do…while 循环统计 word 里 1 的个数，结果与 bit_count 完全一致（含 word == 0 时返
回 0）。 */
uint8_t  bit_count_do_while(uint32_t word);

/* 生成 [hi:lo] 这一段全为 1 的掩码（含两端，共 hi-lo+1 位）。
 * 例：bit_mask_make(7, 4)  -> 0x000000F0
```

```
 *     bit_mask_make(31, 0) -> 0xFFFFFFFF */
uint32_t bit_mask_make(uint8_t hi, uint8_t lo);

/* 取出 word 里 [hi:lo] 这一段的值（已右移到最低位）。
 * 例：bit_field_get(0x12345678, 11, 4) -> 0x67 */
uint32_t bit_field_get(uint32_t word, uint8_t hi, uint8_t lo);

/* 把 value 的低若干位写进 word 的 [hi:lo] 段，其余位保持原值，返回结果。
 * 例：bit_field_set(0xFFFFFFFF, 7, 4, 5) -> 0xFFFFFF5F */
uint32_t bit_field_set(uint32_t word, uint8_t hi, uint8_t lo, uint32_t value);

/* 把一个字节的 8 个二进制位左右颠倒。
 * 例：0x01 -> 0x80，0x0F -> 0xF0，0xA5 -> 0xA5。 */
uint8_t  bit_reverse8(uint8_t byte);

#endif /* BITOPS_H */
```

实现要求

1. 每个返回 `word` 的函数先判位号范围，非法时返回 `word` 原值；`bit_get` / `bit_field_get` / `bit_mask_make` 非法时返回 0。
2. 位运算写 `1u << pos`，不写 `1 << pos`。
3. `bit_mask_make`：`width = hi - lo + 1`；`width >= 32` 时返回 `0xFFFFFFFFu`； 否则返回 `((1u << width) - 1u) << lo`。
4. `bit_count` 用 `while`；`bit_count_do_while` 用 `do…while`。
5. `bit_field_get` 用右移 + 掩码；`bit_field_set` 先清该段再或入 `value` 的有效位。
6. `bit_reverse8` 用循环 8 次实现，不打表。

验收用例

调用	期望结果
`bit_set(0x00000000, 3)`	`0x00000008`
`bit_set(0x00000000, 31)`	`0x80000000`
`bit_set(0x00000000, 32)`	`0x00000000`（非法位号，原样返回）
`bit_clear(0x000000FF, 3)`	`0x000000F7`
`bit_clear(0x80000000, 31)`	`0x00000000`
`bit_toggle(0x00000008, 3)`	`0x00000000`
`bit_toggle(0x00000000, 5)`	`0x00000020`
`bit_get(0x00000008, 3)`	`1`
`bit_get(0x00000008, 2)`	`0`
`bit_get(0x80000000, 31)`	`1`
`bit_count(0xFFFFFFF0)`	`28`

调用	期望结果
`bit_count(0x0F0F0F0F)`	`16`
`bit_count(0)`	`0`
`bit_count_do_while(0xFFFFFFF0)`	`28`
`bit_count_do_while(0)`	`0`
`bit_mask_make(3, 0)`	`0x0000000F`
`bit_mask_make(7, 4)`	`0x000000F0`
`bit_mask_make(4, 3)`	`0x00000018`
`bit_mask_make(31, 0)`	`0xFFFFFFFF`
`bit_mask_make(3, 4)`	`0`（`hi < lo`，非法）
`bit_mask_make(32, 0)`	`0`（`hi > 31`，非法）
`bit_field_get(0x12345678, 11, 4)`	`0x67`
`bit_field_get(0x12345678, 15, 12)`	`0x05`
`bit_field_set(0, 7, 4, 0xA)`	`0x000000A0`
`bit_field_set(0xFFFFFFFF, 7, 4, 5)`	`0xFFFFFF5F`
`bit_reverse8(0x01)`	`0x80`
`bit_reverse8(0x0F)`	`0xF0`
`bit_reverse8(0xA5)`	`0xA5`

模块 2　`checksum` —— 校验值计算

文件：`inc/checksum.h` + `src/checksum.c` **对应 PPT**：第 5 讲（数组名退化成指针、`const` 指针参数）

接口（`inc/checksum.h` 的内容）

```
/* ============================================================================
 * checksum.h —— 校验值计算模块
 *
 * 本模块只做计算，返回值就是校验值本身。
 *
 * 长度约定：长度由调用者单独用 uint32_t 传进来。
 * 形参 data 是指针，函数里 sizeof(data) 得到的是指针大小，不是缓冲区长度。
 *
 * 参数约定：写成 const uint8_t *，表示本函数只读，不修改调用者的数据。
 *
 * data == NULL 或 len == 0 时返回 0。
 * ============================================================================ */
#ifndef CHECKSUM_H
#define CHECKSUM_H
```

```c
#include <stdint.h>

/* 累加和：所有字节相加后取低 8 位（对 256 取模）。 */
uint8_t cksum_sum8(const uint8_t *data, uint32_t len);

/* 异或校验：所有字节逐字节异或，初值 0。 */
uint8_t cksum_xor8(const uint8_t *data, uint32_t len);

/* ★ 延伸：CRC8
 * 多项式 0x07，初值 0x00，不反转，异或输出 0x00。
 * 标准值：crc8("123456789", 9) == 0xF4 */
uint8_t crc8(const uint8_t *data, uint32_t len);

/* ★ 延伸：CRC16
 * 多项式 0xA001（反射写法），初值 0xFFFF，异或输出 0x0000。
 * 标准值：crc16("123456789", 9) == 0x4B37 */
uint16_t crc16(const uint8_t *data, uint32_t len);

#endif /* CHECKSUM_H */
```

实现要求

1. `data == NULL` 或 `len == 0` 时返回 0。
2. `cksum_sum8`：用 `uint32_t` 累加所有字节，最后 `& 0xFF`。
3. `cksum_xor8`：初值 0，逐字节异或。
4. 循环变量用 `uint32_t i`。
5. ★ `crc8`：`crc = 0`；每字节先 `crc ^= data[i]`，再循环 8 次： 最高位为 1 时 `crc = (crc << 1) ^ 0x07`，否则 `crc <<= 1`；全程用 `uint8_t`。
6. ★ `crc16`：`crc = 0xFFFF`；每字节先 `crc ^= data[i]`，再循环 8 次： 最低位为 1 时 `crc = (crc >> 1) ^ 0xA001`，否则 `crc >>= 1`。

验收用例

调用	期望结果
`cksum_sum8({0x78,0x56,0x34,0x12}, 4)`	`0x14`
`cksum_xor8({0x78,0x56,0x34,0x12}, 4)`	`0x08`
`cksum_sum8(NULL, 4)`	`0`
`cksum_xor8(d, 0)`	`0`
`cksum_sum8("1", 1)`	`0x31`
★ `crc8("123456789", 9)`	`0xF4`
★ `crc16("123456789", 9)`	`0x4B37`

模块 3　`status_word` —— 32 位状态字的打包与解码

文件：`inc/status_word.h` + `src/status_word.c` **对应 PPT**：第 3 讲（状态字 / 掩码 / 置位清位）、第 5 讲（结构体、指针作输出参数）、第 2 讲（`typedef` / `enum`）

字段定义：

位域	含义	取值范围
`[0]`	使能标志 `enable`	0 / 1
`[3:1]`	错误码 `error`	0 ~ 7
`[11:4]`	温度 `temp_c`（℃）	0 ~ 255
`[15:12]`	运行模式 `mode`	0 ~ 15
`[31:16]`	电压 `volt_mv`（mV）	0 ~ 65535

接口（`inc/status_word.h` 的内容）

```
/* ============================================================================
 * status_word.h —— 32 位状态字的打包与解码
 *
 * 本模块处理的位布局：
 *
 *   位:  31            16 15   12 11        4 3   1 0
 *        +---------------+-------+-----------+-----+-+
 *        |    volt_mv    | mode  |  temp_c   |error|e|
 *        +---------------+-------+-----------+-----+-+
 *         [31:16]         [15:12]  [11:4]     [3:1] [0]
 *
 * 解码 = 把 raw 拆成 5 个字段（纯计算，直接改调用者的结构体）。
 * 打包 = 把 5 个字段合成 raw；字段可能越界，所以返回错误码。
 * ============================================================================ */
#ifndef STATUS_WORD_H
#define STATUS_WORD_H

#include <stdint.h>

/* 每个字段的起始位号 */
#define STATUS_ENABLE_POS   0u    /* 使能标志从第 0 位开始 */
#define STATUS_ERROR_POS    1u    /* 错误码从第 1 位开始 */
#define STATUS_TEMP_POS     4u    /* 温度从第 4 位开始 */
#define STATUS_MODE_POS     12u   /* 运行模式从第 12 位开始 */
#define STATUS_VOLT_POS     16u   /* 电压从第 16 位开始 */

/* 每个字段的掩码：这一段全 1，其他位全 0 */
#define STATUS_ENABLE_MSK   (1u     << STATUS_ENABLE_POS)   /* 0x00000001 */
#define STATUS_ERROR_MSK    (0x7u   << STATUS_ERROR_POS)    /* 0x0000000E */
#define STATUS_TEMP_MSK     (0xFFu  << STATUS_TEMP_POS)     /* 0x00000FF0 */
#define STATUS_MODE_MSK     (0xFu   << STATUS_MODE_POS)     /* 0x0000F000 */
#define STATUS_VOLT_MSK     (0xFFFFu << STATUS_VOLT_POS)    /* 0xFFFF0000 */

/* 返回码（只给打包和字段检查用） */
```

```c
typedef enum {
    STATUS_OK        =  0,   /* 成功 */
    STATUS_ERR_NULL  = -1,   /* 传进来的指针是空指针 */
    STATUS_ERR_PARAM = -2    /* 字段超出取值范围 */
} status_err_t;

/* 拆开之后的结果 */
typedef struct {
    uint8_t  enable;   /* [0]     使能标志，0 或 1 */
    uint8_t  error;    /* [3:1]   错误码，0 ~ 7 */
    uint8_t  temp_c;   /* [11:4]  温度，0 ~ 255，单位 ℃ */
    uint8_t  mode;     /* [15:12] 运行模式，0 ~ 15 */
    uint16_t volt_mv;  /* [31:16] 电压，0 ~ 65535，单位 mV */
} status_fields_t;

/* 解码：把 raw 拆成 5 个字段写到 *fields。纯计算，直接返回 void。
 * 调用者保证 fields 不是空指针。 */
void status_decode(uint32_t raw, status_fields_t *fields);

/* 打包：把 *fields 的 5 个字段拼成一个 32 位状态字写到 *raw。
 *   fields 或 raw 为 NULL -> STATUS_ERR_NULL
 *   字段超出取值范围      -> STATUS_ERR_PARAM，且不修改 *raw
 *   全部合法              -> STATUS_OK */
status_err_t status_encode(const status_fields_t *fields, uint32_t *raw);

/* 检查 5 个字段是否都在取值范围内。
 *   fields == NULL -> STATUS_ERR_NULL
 *   有字段越界     -> STATUS_ERR_PARAM
 *   全部合法       -> STATUS_OK
 * 范围：enable 0~1、error 0~7、mode 0~15；temp_c 和 volt_mv 的类型已限定范围。 */
status_err_t status_fields_check(const status_fields_t *fields);

/* 把 *raw 的 bit0 置 1（on 非 0）或清 0（on 为 0），其他 31 位保持原值。
 * 调用者保证 raw 不是空指针。 */
void status_set_enable(uint32_t *raw, uint8_t on);

/* 按下面格式打印，调用者保证输出可用。 */
void status_print(uint32_t raw);

#endif /* STATUS_WORD_H */
```

实现要求

1. `status_decode`：每个字段先 `&` 掩码，再右移到最低位。
2. `status_fields_check`：`enable > 1`、`error > 7`、`mode > 15` 任一成立返回 `STATUS_ERR_PARAM`。
3. `status_encode`：先调 `status_fields_check`；不通过就返回对应错误码，不修改 `*raw`； 通过后每个字段先 `&` 掩码，再 `<<` 起始位，最后 `|` 起来写回 `*raw`。
4. `status_set_enable`：置位用 `*raw |= STATUS_ENABLE_MSK`，清位用 `*raw &= ~STATUS_ENABLE_MSK`。
5. `status_print`：`raw` 用 `%08X`，其余字段先转成 `uint32_t` 再用 `%u`，格式为 `raw=0x12345678 en=0 err=4 temp=103 mode=5 mv=4660`。

验收用例

调用	期望结果
`status_decode(0x12345678, &f)`	`en=0`、`err=4`、`temp=103`、`mode=5`、`mv=4660`
`status_encode(&f, &raw)`（`f` 为上面那组值）	`STATUS_OK`，`raw = 0x12345678`
`status_encode(&f, &raw)`，其中 `f.error = 8`	`STATUS_ERR_PARAM`，`raw` 不变
`status_encode(NULL, &raw)`	`STATUS_ERR_NULL`
`status_fields_check(&f)`，字段全部合法	`STATUS_OK`
`status_fields_check(&f)`，`f.mode = 16`	`STATUS_ERR_PARAM`
`status_fields_check(NULL)`	`STATUS_ERR_NULL`
`status_set_enable(&raw, 1)`，`raw = 0x00000000`	`raw = 0x00000001`
`status_set_enable(&raw, 0)`，`raw = 0x12345679`	`raw = 0x12345678`
`status_set_enable(&raw, 0)`，`raw = 0xFFFFFFFF`	`raw = 0xFFFFFFFE`
`status_print(0x12345678)`	打印 `raw=0x12345678 en=0 err=4 temp=103 mode=5 mv=4660`

模块 4　`fifo` —— 环形缓冲区

文件：`inc/fifo.h` + `src/fifo.c` **对应 PPT**：第 5 讲（数组、结构体、指针作输出参数）、第 2 讲（传值调用 vs 指针、`enum`）

接口（`inc/fifo.h` 的内容）

```
/* ============================================================================
 * fifo.h —— 环形缓冲区
 *
 * 一个 fifo_t 就是一个完整的队列对象，可以定义多个实例。
 *
 * 缓冲区布局（容量 8 时）：
 *
 *     head              tail
 *       v                 v
 *     +----+----+----+----+----+----+----+----+
 *     | 4  | 5  | 6  |    |    |    | A0 | A1 |
 *     +----+----+----+----+----+----+----+----+
 *
 *     head：下一个要读的位置，0 ~ FIFO_CAPACITY-1
 *     tail：下一个要写的位置，0 ~ FIFO_CAPACITY-1
```

```
 *     count：当前元素个数，0 ~ FIFO_CAPACITY；空、满、长度都以它为准
 *
 * 入队可能因为队满而失败、出队可能因为队空而失败，所以这两个函数返回错误码；
 * 查询类的函数直接返回查询结果。
 * ============================================================================ */
#ifndef FIFO_H
#define FIFO_H

#include <stdint.h>

#define FIFO_CAPACITY 16u

/* 返回码（只给入队、出队用） */
typedef enum {
    FIFO_OK        =  0,   /* 成功 */
    FIFO_ERR_NULL  = -1,   /* 指针是空指针 */
    FIFO_ERR_EMPTY = -2,   /* 队列为空 */
    FIFO_ERR_FULL  = -3    /* 队列已满 */
} fifo_err_t;

typedef struct {
    uint8_t  buf[FIFO_CAPACITY];  /* 数据区，环形使用 */
    uint32_t head;                /* 下一个要读的位置 */
    uint32_t tail;                /* 下一个要写的位置 */
    uint32_t count;               /* 当前元素个数 */
} fifo_t;

/* 复位：head、tail、count 全部清零。调用者保证 fifo 不是空指针。 */
void fifo_init(fifo_t *fifo);

/* 队列是否为空：空返回 1，非空返回 0。fifo 为空指针时返回 1。 */
uint8_t  fifo_is_empty(const fifo_t *fifo);

/* 队列是否已满：满返回 1，未满返回 0。fifo 为空指针时返回 0。 */
uint8_t  fifo_is_full(const fifo_t *fifo);

/* 当前元素个数。fifo 为空指针时返回 0。 */
uint32_t fifo_count(const fifo_t *fifo);

/* 入队：把 byte 放到队尾。
 *   fifo == NULL -> FIFO_ERR_NULL
 *   队列已满     -> FIFO_ERR_FULL（不覆盖旧数据）
 *   成功         -> FIFO_OK */
fifo_err_t fifo_push(fifo_t *fifo, uint8_t byte);

/* 出队：把队头字节写到 *out。
 *   fifo 或 out 为 NULL -> FIFO_ERR_NULL
 *   队列为空            -> FIFO_ERR_EMPTY（不修改 *out）
 *   成功                -> FIFO_OK */
fifo_err_t fifo_pop(fifo_t *fifo, uint8_t *out);

/* 一次最多取出 max_len 个字节，按先进先出顺序写到 out[0]、out[1]……
 * 返回实际取出的个数（队列为空时返回 0），剩下的留在队列里。
 * 调用者保证 fifo 和 out 不是空指针。 */
```

```
uint32_t fifo_drain(fifo_t *fifo, uint8_t *out, uint32_t max_len);

#endif /* FIFO_H */
```

实现要求

1. 空、满、长度全部以 `count` 为准。
2. `fifo_init`：`head`、`tail`、`count` 清零。
3. `fifo_push`：判空 → 判满 → 写 `buf[tail]` → `tail = (tail + 1) % FIFO_CAPACITY` → `count++`。
4. `fifo_pop`：判空 → 判空队列 → 读 `buf[head]` 到 `*out` → `head` 前进 → `count--`。
5. `fifo_drain`：循环调用 `fifo_pop`，取到 `max_len` 个或队列空为止，返回实际取出的个数。
6. `fifo_is_empty`、`fifo_is_full`、`fifo_count` 的参数用 `const fifo_t *`。

验收用例

场景	期望结果
新队列 `fifo_is_empty(&q)`	`1`
新队列 `fifo_is_full(&q)`	`0`
新队列 `fifo_count(&q)`	`0`
空队列 `fifo_pop(&q, &v)`	`FIFO_ERR_EMPTY`
`fifo_push(NULL, 1)`	`FIFO_ERR_NULL`
`fifo_pop(&q, NULL)`	`FIFO_ERR_NULL`
依次 push `0x11`、`0x22`、`0x33`	都返回 `FIFO_OK`；`fifo_count` → `3`
第一次 `pop`	`FIFO_OK`，`v = 0x11`
装满 16 个后再 push	`FIFO_ERR_FULL`；`fifo_is_full` → `1`
取走 4 个、再放 4 个（绕回缓冲区开头），然后 `fifo_drain(&q, out, 32)`	`16`；`out[0]=4`、`out[11]=15`、`out[12]=0xA0`、`out[15]=0xA3`
队列里有 3 个，`fifo_drain(&q, out, 2)`	`2`；之后 `fifo_count` → `1`，再 `pop` 得到 `3`
空队列 `fifo_drain(&q, out, 32)`	`0`

绕回用例的操作序列：`push 0..15` → `pop` 4 次（`head` 变 4）→ `push 0xA0..0xA3`（`tail` 从 0 走到 4），此时队列内容为 `4,5,…,15, 0xA0,0xA1,0xA2,0xA3`。

模块 5　`pid` —— PID 控制器

文件：`inc/pid.h` + `src/pid.c` **对应 PPT**：第 2 讲（函数 / 作用域 / `static`）、第 1 讲（`float`、运算符优先级）、第 5 讲（结构体指针）

> 本模块涉及小数运算，参数和返回值用 `float`。

接口（`inc/pid.h` 的内容）

```c
/* ============================================================================
 * pid.h —— PID 控制器模块
 *
 * 模块的内部状态全部放进 pid_t，由调用者持有，可定义多个实例。
 * 对外只暴露下面 5 个函数，内部辅助函数一律用 static，不写进头文件。
 *
 * 本模块的参数设置和计算都是纯计算，返回值就是计算结果或 void，不使用错误码。
 * ============================================================================ */
#ifndef PID_H
#define PID_H

#include <stdint.h>

#define PID_OUT_MAX_DEFAULT    1000.0f
#define PID_OUT_MIN_DEFAULT  (-1000.0f)

typedef struct {
    float   kp;          /* 比例增益 */
    float   ki;          /* 积分增益 */
    float   kd;          /* 微分增益 */
    float   integral;    /* 积分累积量 */
    float   prev_error;  /* 上一次的误差 */
    float   out_max;     /* 输出上限 */
    float   out_min;     /* 输出下限 */
    uint8_t first_run;   /* 1 = 还没算过；0 = 已经算过 */
} pid_t;

/* 三个增益清 0，限幅设为默认值，内部状态复位。调用者保证 pid 不是空指针。 */
void  pid_init(pid_t *pid);

/* 复位 integral、prev_error、first_run，保留增益和限幅。 */
void  pid_reset(pid_t *pid);

/* 设置三个增益。 */
void  pid_set_gains(pid_t *pid, float kp, float ki, float kd);

/* 设置输出限幅；out_min > out_max 时先交换再存。 */
void  pid_set_limit(pid_t *pid, float out_min, float out_max);

/* 算一步，返回限幅后的输出。执行顺序：
 *   1. dt <= 0.0f -> 返回 0.0f，不改变任何内部状态
 *   2. error = setpoint - measured
 *   3. integral += error * dt
 *   4. derivative = (error - prev_error) / dt；第一次调用时 derivative = 0
 *   5. output = kp*error + ki*integral + kd*derivative
 *   6. output 限幅到 [out_min, out_max]，作为返回值
 *   7. prev_error = error，first_run = 0
 * 调用者保证 pid 不是空指针。 */
float pid_calculate(pid_t *pid, float setpoint, float measured, float dt);

#endif /* PID_H */
```

实现要求

1. 按上面注释里的 7 步顺序实现 `pid_calculate`，第 1 步在任何计算之前判断。
2. `pid_init`：三个增益清 0，限幅设为默认值，调用 `pid_reset`。
3. `pid_set_limit`：`out_min > out_max` 时先交换再存。
4. 只给自己用的小工具函数（如限幅 `pid_clamp`）必须加 `static`。
5. 浮点常量带 `f` 后缀（`0.0f`）。

验收用例

增益 `kp=2.0, ki=0.5, kd=0.1`，`dt=0.1`：

第几步	setpoint	measured	期望返回值
1	100	80	`41.000`
2	100	90	`11.500`
3	100	100	`-8.500`
4	100	120	`-59.500`

场景	期望结果
`pid_init(&pid)` 之后	`kp=0`、`out_max=1000`、`out_min=-1000`、`first_run=1`
`kp=1, ki=0, kd=0`，算 `(50, 30, 0.1)`	`20.000`
`kp=100`，算 `(100, 0, 0.1)`	`1000.000`
紧接着算 `(0, 100, 0.1)`	`-1000.000`
先正常算一步，再传 `dt = 0`	返回 `0.000`，`integral`、`prev_error`、`first_run` 都不变
`pid_set_limit(&pid, 500, -500)`	`out_min = -500`、`out_max = 500`
`pid_reset(&pid)`	`integral = 0`、`first_run = 1`，增益保留

模块 6　`lowpass` —— 一阶低通滤波器

文件：`inc/lowpass.h` + `src/lowpass.c` **对应 PPT**：第 5 讲（结构体指针）、第 2 讲（`typedef` / `enum`）、第 1 讲（`float`）

> 本模块涉及小数运算，参数和返回值用 `float`。

接口（`inc/lowpass.h` 的内容）

```
/* ============================================================================
 * lowpass.h —— 一阶低通滤波器（递推式）
 *
 * 滤波公式（alpha 是滤波系数，x 是本次采样值，y 是上一次的输出）：
 *   第一次调用：y = x
 *   之后每次：   y = y + alpha * (x - y)
 *
```

```
 * alpha 越接近 1 跟随越快、滤波越弱；越接近 0 越平滑、滞后越大。
 * ========================================================================== */
#ifndef LOWPASS_H
#define LOWPASS_H

#include <stdint.h>

/* 返回码（只给初始化用：alpha 需要校验） */
typedef enum {
    LOWPASS_OK        =  0,   /* 成功 */
    LOWPASS_ERR_NULL  = -1,   /* 指针是空指针 */
    LOWPASS_ERR_PARAM = -2    /* alpha 不在 0 < alpha <= 1 范围内 */
} lowpass_err_t;

typedef struct {
    float   alpha;      /* 滤波系数，0 < alpha <= 1 */
    float   y;          /* 上一次的滤波输出 */
    uint8_t first_run;  /* 1 = 还没算过；0 = 已经算过 */
} lowpass_t;

/* 初始化：alpha 存入结构体，y 清 0，first_run 置 1。
 *   lp == NULL                -> LOWPASS_ERR_NULL
 *   alpha <= 0 或 alpha > 1   -> LOWPASS_ERR_PARAM（不修改结构体）
 *   成功                      -> LOWPASS_OK */
lowpass_err_t lowpass_init(lowpass_t *lp, float alpha);

/* 复位：y 清 0，first_run 置 1，保留 alpha。
 * 调用者保证 lp 不是空指针。 */
void lowpass_reset(lowpass_t *lp);

/* 送入一个采样值，返回滤波输出。
 *   first_run == 1: y = x，并把 first_run 置 0
 *   否则:           y = y + alpha * (x - y)
 * lp 为空指针时返回 0.0f。 */
float lowpass_update(lowpass_t *lp, float x);

#endif /* LOWPASS_H */
```

实现要求

1. `lowpass_init`：先判 `lp == NULL` → `LOWPASS_ERR_NULL`； 再判 `alpha <= 0.0f || alpha > 1.0f` → `LOWPASS_ERR_PARAM`（不修改结构体）； 通过后写入 `alpha`，`y` 清 0，`first_run` 置 1，返回 `LOWPASS_OK`。
2. `lowpass_update`：按注释里的公式计算并返回；每次都要更新 `y` 和 `first_run`；`lp == NULL` 返回 `0.0f`。
3. `lowpass_reset`：只改 `y` 和 `first_run`，不动 `alpha`。
4. 递推式写成 `y = y + alpha * (x - y)`。

验收用例

```
lowpass_t lp;
lowpass_t lp2;
```

调用	期望结果
`lowpass_init(NULL, 0.5f)`	`LOWPASS_ERR_NULL`
`lowpass_init(&lp2, 0.0f)`	`LOWPASS_ERR_PARAM`
`lowpass_init(&lp2, -0.1f)`	`LOWPASS_ERR_PARAM`
`lowpass_init(&lp2, 1.5f)`	`LOWPASS_ERR_PARAM`
`lowpass_init(&lp2, 1.0f)`	`LOWPASS_OK`（边界值合法）
`lowpass_init(&lp, 0.5f)`	`LOWPASS_OK`
`lowpass_update(&lp, 10.0f)`（第一次）	`10.000`
`lowpass_update(&lp, 20.0f)`	`15.000`
`lowpass_update(&lp, 30.0f)`	`22.500`
`lowpass_update(&lp, 30.0f)`	`26.250`
`lowpass_reset(&lp)` 后 `lowpass_update(&lp, 8.0f)`	`8.000`
`lowpass_update(NULL, 1.0f)`	`0.000`

`alpha = 0.25` 时的递推结果：

调用	期望结果
`lowpass_init(&lp, 0.25f)` 后 `lowpass_update(&lp, 100.0f)`	`100.000`
`lowpass_update(&lp, 200.0f)`	`125.000`
`lowpass_update(&lp, 200.0f)`	`143.750`

模块 7　`moving_avg` —— 滑动平均滤波器

文件：`inc/moving_avg.h` + `src/moving_avg.c` **对应 PPT**：第 5 讲（数组、结构体、指针）、第 2 讲（`static`、`typedef`）

接口（`inc/moving_avg.h` 的内容）

```
/* ============================================================================
 * moving_avg.h —— 滑动平均滤波器
 *
 * 维护一个长度为 MAVG_WINDOW 的滑动窗口：每来一个新数据，窗口未满时直接填入，
 * 窗口已满时覆盖最旧的数据，然后返回窗口内数据的平均值。
 * 平均值用整数除法，向零截断。
```

```
 *
 * 缓冲区布局（MAVG_WINDOW = 4，已填入 10、20、30 时）：
 *
 *     index
 *       v
 *     +----+----+----+----+
 *     | 10 | 20 | 30 |    |
 *     +----+----+----+----+
 *     sum = 60，count = 3，下一个写到 index = 3
 * ============================================================================ */
#ifndef MOVING_AVG_H
#define MOVING_AVG_H

#include <stdint.h>

#define MAVG_WINDOW 4u

typedef struct {
    int32_t  buf[MAVG_WINDOW];  /* 环形数据区 */
    int32_t  sum;               /* 窗口内数据之和 */
    uint32_t index;             /* 下一个要写的位置，0 ~ MAVG_WINDOW-1 */
    uint32_t count;             /* 已填入的数据个数，0 ~ MAVG_WINDOW */
} mavg_t;

/* 复位：sum、index、count 全部清零。调用者保证 mavg 不是空指针。 */
void mavg_init(mavg_t *mavg);

/* 送入一个采样值，返回窗口内数据的平均值。
 *   count <  MAVG_WINDOW：buf[index] = x，sum += x，count++
 *   count == MAVG_WINDOW：sum -= buf[index]，buf[index] = x，sum += x
 *   两种情况都要把 index 环形前进
 * 返回值 = sum / (int32_t)count（整数除法，向零截断）。
 * mavg 为空指针时返回 0。 */
int32_t mavg_update(mavg_t *mavg, int32_t x);

/* 当前窗口内的数据个数（0 ~ MAVG_WINDOW）。mavg 为空指针时返回 0。 */
uint32_t mavg_count(const mavg_t *mavg);

#endif /* MOVING_AVG_H */
```

实现要求

1. `mavg_update`：`mavg == NULL` 返回 0；否则按注释里的两条路径更新 `buf`、`sum`、`count`、`index`。
2. 窗口已满时先做 `sum -= buf[index]` 再写新值，保证 `sum` 始终等于窗口内数据之和。
3. `index` 前进用 `(index + 1u) % MAVG_WINDOW`。
4. 返回 `sum / (int32_t)count`，整数除法向零截断。
5. `mavg_count` 的参数用 `const mavg_t *`。

验收用例

```
mavg_t m;
```

调用	期望结果
`mavg_init(&m)` 后 `mavg_count(&m)`	`0`
`mavg_update(&m, 10)`	`10`（窗口 {10}，10/1）
`mavg_update(&m, 20)`	`15`（窗口 {10,20}，30/2）
`mavg_update(&m, 30)`	`20`（窗口 {10,20,30}，60/3）
`mavg_update(&m, 40)`	`25`（窗口 {10,20,30,40}，100/4）
`mavg_update(&m, 50)`	`35`（挤出 10：140/4）
`mavg_update(&m, 60)`	`45`（挤出 20：180/4）
`mavg_update(&m, 70)`	`55`（挤出 30：220/4）
`mavg_update(&m, 80)`	`65`（挤出 40：260/4）
`mavg_count(&m)`	`4`
`mavg_update(NULL, 1)`	`0`
`mavg_count(NULL)`	`0`

整数截断和负数的行为：

调用	期望结果
`mavg_init(&m)` 后 `mavg_update(&m, -10)`	`-10`（-10/1）
`mavg_update(&m, 5)`	`-2`（-5/2，向零截断）
`mavg_update(&m, 5)`	`0`（0/3）
`mavg_update(&m, 5)`	`1`（5/4）

模块 8　`median` —— 中值滤波

文件：`inc/median.h` + `src/median.c` **对应 PPT**：第 5 讲（数组、结构体、指针）、第 1 讲（嵌套循环）

接口（`inc/median.h` 的内容）

```
/* ============================================================================
 * median.h —— 中值滤波
 *
 * 维护一个长度为 MEDIAN_WINDOW 的滑动窗口：每来一个新数据，窗口未满时填入，
 * 窗口已满时覆盖最旧的数据，然后对窗口内的有效数据排序，
 * 返回排序后下标为 count/2 的元素。
 *
```

```
 * 缓冲区布局：窗口未满时有效数据在 buf[0..count-1]；窗口已满时全部元素有效。
 * ========================================================================== */
#ifndef MEDIAN_H
#define MEDIAN_H

#include <stdint.h>

#define MEDIAN_WINDOW 5u

typedef struct {
    uint16_t buf[MEDIAN_WINDOW];  /* 数据区 */
    uint32_t index;               /* 下一个要写的位置，0 ~ MEDIAN_WINDOW-1 */
    uint32_t count;               /* 已填入的数据个数，0 ~ MEDIAN_WINDOW */
} median_t;

/* 复位：index、count 全部清零。调用者保证 median 不是空指针。 */
void median_init(median_t *median);

/* 送入一个采样值，返回当前窗口的中值。
 *   count <  MEDIAN_WINDOW：buf[index] = x，count++
 *   count == MEDIAN_WINDOW：buf[index] = x（覆盖最旧数据）
 *   两种情况都要把 index 环形前进
 * 然后把 buf[0..count-1] 复制到局部数组排序，返回下标 count/2 的元素。
 * median 为空指针时返回 0。 */
uint16_t median_update(median_t *median, uint16_t x);

/* 当前窗口内的数据个数（0 ~ MEDIAN_WINDOW）。median 为空指针时返回 0。 */
uint32_t median_count(const median_t *median);

#endif /* MEDIAN_H */
```

实现要求

1. `median_update`：`median == NULL` 返回 0；否则先把新数据写入 `buf[index]`，更新 `count`， 再把 `index` 环形前进。
2. 把 `buf[0..count-1]` 复制到局部数组 `uint16_t tmp[MEDIAN_WINDOW]`，**不要破坏 `buf` 的环形顺序**。
3. 用选择排序或冒泡排序把 `tmp` 的前 `count` 个元素升序排好。
4. 返回 `tmp[count / 2]`。
5. `median_count` 的参数用 `const median_t *`。

验收用例

```
median_t md;
```

调用	期望结果
`median_init(&md)` 后 `median_count(&md)`	`0`
`median_update(&md, 10)`	`10`（窗口 {10}）

调用	期望结果
`median_update(&md, 30)`	`30`（排序 {10,30}，下标 1）
`median_update(&md, 20)`	`20`（排序 {10,20,30}，下标 1）
`median_update(&md, 50)`	`30`（排序 {10,20,30,50}，下标 2）
`median_update(&md, 40)`	`30`（排序 {10,20,30,40,50}，下标 2）
`median_update(&md, 100)`	`40`（覆盖 10，排序 {20,30,40,50,100}，下标 2）
`median_update(&md, 20)`	`40`（覆盖 30，排序 {20,20,40,50,100}，下标 2）
`median_count(&md)`	`5`
`median_update(NULL, 1)`	`0`
`median_count(NULL)`	`0`

模块 9　`slew` —— 变化率限制器

文件：`inc/slew.h` + `src/slew.c` **对应 PPT**：第 5 讲（结构体指针）、第 2 讲（`typedef` / `enum`）、第 1 讲（`float`、`if…else`）

> 本模块涉及小数运算，参数和返回值用 `float`。

接口（`inc/slew.h` 的内容）

```c
/* ============================================================================
 * slew.h —— 变化率限制器
 *
 * 把输入的突变限制成每步只变化一小段：一个时间步 dt 内允许的最大变化量为
 *   上升：rise_max * dt
 *   下降：fall_max * dt
 *
 * 第一次调用直接到位（不做限制），之后每次按上面的限制逼近目标值。
 * ============================================================================ */
#ifndef SLEW_H
#define SLEW_H

#include <stdint.h>

/* 返回码（只给初始化用：两个速率需要校验） */
typedef enum {
    SLEW_OK        =  0,   /* 成功 */
    SLEW_ERR_NULL  = -1,   /* 指针是空指针 */
    SLEW_ERR_PARAM = -2    /* rise_max 或 fall_max 不是正数 */
} slew_err_t;

typedef struct {
    float   rise_max;   /* 每单位时间允许的最大上升量，必须 > 0 */
    float   fall_max;   /* 每单位时间允许的最大下降量，必须 > 0 */
```

```
    float   out;        /* 上一次的输出 */
    uint8_t first_run;  /* 1 = 还没算过；0 = 已经算过 */
} slew_t;

/* 初始化：两个速率存入结构体，out 清 0，first_run 置 1。
 *   slew == NULL                    -> SLEW_ERR_NULL
 *   rise_max <= 0 或 fall_max <= 0 -> SLEW_ERR_PARAM（不修改结构体）
 *   成功                            -> SLEW_OK */
slew_err_t slew_init(slew_t *slew, float rise_max, float fall_max);

/* 复位：out 清 0，first_run 置 1，保留两个速率。
 * 调用者保证 slew 不是空指针。 */
void slew_reset(slew_t *slew);

/* 送入目标值和本次的时间步长，返回限制后的输出。
 *   1. slew == NULL 或 dt <= 0.0f -> 返回 0.0f，不改变内部状态
 *   2. first_run == 1             -> out = target，first_run 置 0
 *   3. delta = target - out
 *      delta >  rise_max * dt      -> out += rise_max * dt
 *      delta < -(fall_max * dt)    -> out -= fall_max * dt
 *      否则                        -> out = target */
float slew_update(slew_t *slew, float target, float dt);

#endif /* SLEW_H */
```

实现要求

1. `slew_init`：先判 `slew == NULL` → `SLEW_ERR_NULL`； 再判 `rise_max <= 0.0f || fall_max <= 0.0f` → `SLEW_ERR_PARAM`（不修改结构体）； 通过后写入两个速率，`out` 清 0，`first_run` 置 1，返回 `SLEW_OK`。
2. `slew_update`：按注释里的 4 步顺序实现；第 1 步必须在任何计算之前判断。
3. `slew_reset`：只改 `out` 和 `first_run`，不动两个速率。
4. 上升和下降分别用 `rise_max * dt` 和 `fall_max * dt`，不要把 `fall_max` 当成负数。

验收用例

```
slew_t sl;
slew_t sl2;
```

调用	期望结果
`slew_init(NULL, 100.0f, 200.0f)`	`SLEW_ERR_NULL`
`slew_init(&sl2, 0.0f, 200.0f)`	`SLEW_ERR_PARAM`
`slew_init(&sl2, 100.0f, -1.0f)`	`SLEW_ERR_PARAM`
`slew_init(&sl, 100.0f, 200.0f)`	`SLEW_OK`
`slew_update(&sl, 500.0f, 0.1f)`（第一次）	`500.000`（直接到位）

调用	期望结果
`slew_update(&sl, 0.0f, 0.1f)`	`480.000`（下降被限到 20）
`slew_update(&sl, 0.0f, 0.1f)`	`460.000`
`slew_update(&sl, 1000.0f, 0.1f)`	`470.000`（上升被限到 10）
`slew_update(&sl, 471.0f, 0.1f)`	`471.000`（差 1，在允许范围内，直接到位）
`slew_update(&sl, 471.0f, 0.0f)`	`0.000`，`out` 保持 `471.000`
`slew_update(&sl, 471.0f, -0.1f)`	`0.000`，`out` 保持 `471.000`
`slew_reset(&sl)` 后 `slew_update(&sl, 900.0f, 0.1f)`	`900.000`
`slew_update(NULL, 1.0f, 0.1f)`	`0.000`

模块 10　`frame` —— 通信帧的解析与打包

文件：`inc/frame.h` + `src/frame.c` **对应 PPT**：第 5 讲综合（数组 / 指针 / 结构体 / `memcpy` / 指针算术与比较）、第 1 讲（`break` / `continue`）

帧格式（`LEN` 是 `DATA` 的字节数，取值 0 ~ 8）：

```
字节偏移:   0      1      2      3              3+LEN      3+LEN+1
          +------+------+------+--------------+----------+
          | SOF  | LEN  | CMD  |   DATA[0..]  |   CRC    |
          | 0xAA | 1 B  | 1 B  |    LEN B     |   1 B    |
          +------+------+------+--------------+----------+

总长度 = LEN + 4
CRC8 的计算范围 = LEN、CMD、DATA，即从字节 1 开始的 (2 + LEN) 个字节
```

接口（`inc/frame.h` 的内容）

```
/* ============================================================================
 * frame.h —— 通信帧的解析与打包
 *
 * 帧格式（LEN 是 DATA 的字节数，取值 0 ~ 8）：
 *
 *   字节偏移:   0      1      2      3              3+LEN      3+LEN+1
 *             +------+------+------+--------------+----------+
 *             | SOF  | LEN  | CMD  |   DATA[0..]  |   CRC    |
 *             | 0xAA | 1 B  | 1 B  |    LEN B     |   1 B    |
 *             +------+------+------+--------------+----------+
 *
 *   总长度 = LEN + 4
 *   CRC8 的计算范围 = LEN、CMD、DATA，即从字节 1 开始的 (2 + LEN) 个字节
 *
```

```
 * 解析、打包、扫描都要做长度和校验判断，失败原因需要区分，所以返回错误码。
 * 解析时必须逐字段搬运（memcpy），不得把字节流强转成 frame_t *。
 * ============================================================================ */
#ifndef FRAME_H
#define FRAME_H

#include <stdint.h>

#define FRAME_SOF        0xAA                       /* 帧头字节 */
#define FRAME_DATA_MAX   8u                         /* DATA 字段最多 8 字节 */
#define FRAME_OVERHEAD   4u                         /* SOF + LEN + CMD + CRC */
#define FRAME_BUF_MAX    (FRAME_DATA_MAX + FRAME_OVERHEAD)

/* 返回码 */
typedef enum {
    FRAME_OK         =  0,    /* 成功 */
    FRAME_ERR_NULL   = -1,    /* 指针是空指针 */
    FRAME_ERR_SOF    = -2,    /* 帧头不是 0xAA */
    FRAME_ERR_LEN    = -3,    /* 长度字段非法，或缓冲区装不下这一帧 */
    FRAME_ERR_CRC    = -4,    /* 校验不通过 */
    FRAME_ERR_EMPTY = -5     /* 扫描结束仍未找到完整帧 */
} frame_err_t;

typedef struct {
    uint8_t cmd;                  /* CMD 字段：命令码 */
    uint8_t len;                  /* LEN 字段：data 里有效的字节数，0 ~ 8 */
    uint8_t data[FRAME_DATA_MAX]; /* DATA 字段：只有前 len 个字节有效 */
} frame_t;

/* 解析一帧。buf 指向帧首，buf_len 是从帧首开始剩余的字节数。
 * 判断顺序：
 *   buf 或 out 为 NULL              -> FRAME_ERR_NULL
 *   buf_len < FRAME_OVERHEAD        -> FRAME_ERR_LEN
 *   buf[0] != FRAME_SOF             -> FRAME_ERR_SOF
 *   LEN 超出 0 ~ FRAME_DATA_MAX     -> FRAME_ERR_LEN
 *   buf_len < LEN + FRAME_OVERHEAD  -> FRAME_ERR_LEN
 *   CRC8 不匹配                     -> FRAME_ERR_CRC
 *   全部通过                        -> FRAME_OK，填好 *out
 * 成功时填 out->len、out->cmd、out->data[0..len-1]；失败时不修改 *out。 */
frame_err_t frame_parse(const uint8_t *buf, uint32_t buf_len, frame_t *out);

/* 打包一帧，写入的字节数（LEN + 4）写到 *out_len。
 *   frame、buf、out_len 任一为 NULL        -> FRAME_ERR_NULL
 *   frame->len > FRAME_DATA_MAX            -> FRAME_ERR_LEN
 *   buf_size < frame->len + FRAME_OVERHEAD -> FRAME_ERR_LEN
 *   成功                                   -> FRAME_OK
 * 失败时不写 buf，也不修改 *out_len。 */
frame_err_t frame_build(const frame_t *frame, uint8_t *buf, uint32_t buf_size,
                        uint32_t *out_len);

/* 在缓冲区里从头找第一帧能成功解析的数据。
 *   buf、out、out_offset 任一为 NULL -> FRAME_ERR_NULL
 *   找到                             -> FRAME_OK，*out_offset 为帧首相对 buf 的字节
偏移，*out 填好
```

```
 *   扫描结束仍未找到                    -> FRAME_ERR_EMPTY
 *
 * 实现要求：
 *   - 用两个指针 p / end 控制循环边界，用指针比较判断是否走到头；
 *   - 当前位置不是帧头时用 continue 跳过，往后挪一个字节继续找；
 *   - 某个 0xAA 后面这一帧不合格时也继续往后找，不得直接返回失败；
 *   - *out_offset 只在成功时写。 */
frame_err_t frame_scan(const uint8_t *buf, uint32_t buf_len,
                       uint32_t *out_offset, frame_t *out);

/* 按下面格式打印一帧（每个数据字节 %02X 后跟一个空格）：
 *   cmd=0x01 len=3 data=11 22 33
 * 调用者保证 frame 不是空指针。 */
void frame_print(const frame_t *frame);

#endif /* FRAME_H */
```

实现要求

`frame_parse`：

1. `buf == NULL || out == NULL` → `FRAME_ERR_NULL`
2. `buf_len < FRAME_OVERHEAD` → `FRAME_ERR_LEN`
3. `buf[0] != FRAME_SOF` → `FRAME_ERR_SOF`
4. `data_len = buf[1]`；`data_len > FRAME_DATA_MAX` → `FRAME_ERR_LEN`
5. `buf_len < data_len + FRAME_OVERHEAD` → `FRAME_ERR_LEN`
6. 填 `out->len`、`out->cmd = buf[2]`；`data_len > 0` 时 `memcpy(out->data, &buf[3], data_len)`
7. `crc8(&buf[1], 2 + data_len)` 与 `buf[3 + data_len]` 不等 → `FRAME_ERR_CRC`
8. 返回 `FRAME_OK`

`frame_build`：

1. 判空、判 `frame->len > FRAME_DATA_MAX`、判 `buf_size` 是否足够，任一不满足返回对应错误码
2. 写 `buf[0] = FRAME_SOF`、`buf[1] = len`、`buf[2] = cmd`，`memcpy` 数据到 `&buf[3]`
3. `buf[3 + len] = crc8(&buf[1], 2 + len)`；`*out_len = len + 4`；返回 `FRAME_OK`

`frame_scan`：

```
const uint8_t *p;       /* 当前检查的位置 */
const uint8_t *end;     /* buf + buf_len */

/* 先判空 */

end = buf + buf_len;
for (p = buf; p + FRAME_OVERHEAD <= end; p++) {
    if (*p != FRAME_SOF) {
        continue;
    }
    if (frame_parse(p, (uint32_t)(end - p), out) == FRAME_OK) {
        *out_offset = (uint32_t)(p - buf);
```

```
            return FRAME_OK;
        }
    }
    return FRAME_ERR_EMPTY;
```

`frame_print` 输出格式：

```
cmd=0x01 len=3 data=11 22 33
```

验收用例

测试用帧（CRC 已算好）：

```
AA 03 01 11 22 33 64
```

场景	期望结果
`frame_parse(AA 03 01 11 22 33 64, 7, &f)`	`FRAME_OK`，`cmd=0x01`、`len=3`、`data={0x11,0x22,0x33}`
同样数据，`buf_len = 3`	`FRAME_ERR_LEN`
`buf[0]` 改为 `0x55`	`FRAME_ERR_SOF`
最后一个字节取反	`FRAME_ERR_CRC`
`buf[1]` 改为 `0x09`	`FRAME_ERR_LEN`
`buf_len = 6`	`FRAME_ERR_LEN`
`frame_parse(NULL, 7, &f)` / `frame_parse(data, 7, NULL)`	`FRAME_ERR_NULL`
`frame_build(&g, buf, 8, &n)`	`FRAME_OK`，`n = 7`，结果与 `AA 03 01 11 22 33 64` 逐字节相同，第 8 个字节不被写
`frame_build(&g, buf, 6, &n)`	`FRAME_ERR_LEN`，`n` 不变
`frame_build`，`len = 9`	`FRAME_ERR_LEN`
`frame_build(NULL, buf, 7, &n)`	`FRAME_ERR_NULL`
`frame_scan`：2 字节垃圾 + 假帧头 `AA 03 FF FF FF FF` + 真帧	`FRAME_OK`，`*out_offset = 8`
`frame_scan`，`buf_len = 3`	`FRAME_ERR_EMPTY`
`frame_scan`，15 个字节全为 `0x11`	`FRAME_ERR_EMPTY`，`*out_offset` 不变
`frame_scan(NULL, 15, &off, &f)`	`FRAME_ERR_NULL`

场景	期望结果
`frame_print(&g)`	打印 `cmd=0x01 len=3 data=11 22 33`

模块 11　`state_machine` —— 状态机

文件：`inc/state_machine.h` + `src/state_machine.c` **对应 PPT**：第 3 讲（`switch case`）、第 2 讲（`enum`）、第 5 讲（结构体数组 / 二维数组）

迁移规则：

```
IDLE   --ARM-->   ARMED        IDLE   --FAULT--> FAULT
ARMED  --START--> RUN          ARMED  --STOP-->  IDLE      ARMED --FAULT--> FAULT
RUN    --STOP-->  IDLE         RUN    --FAULT--> FAULT
FAULT  --CLEAR--> IDLE

未列出的 (状态, 事件) 组合保持原状态不变
```

接口（`inc/state_machine.h` 的内容）

```c
/* ============================================================================
 * state_machine.h —— 状态机模块
 *
 *   状态(state): 系统当前所处的情况
 *   事件(event): 刚刚发生的动作
 *   迁移(transition): 在某个状态下收到某个事件后转到哪个状态
 *
 * 迁移规则:
 *   IDLE   --ARM-->   ARMED      IDLE   --FAULT--> FAULT
 *   ARMED  --START--> RUN        ARMED  --STOP-->  IDLE      ARMED --FAULT--> FAULT
 *   RUN    --STOP-->  IDLE       RUN    --FAULT--> FAULT
 *   FAULT  --CLEAR--> IDLE
 *   未列出的 (状态, 事件) 组合保持原状态不变。
 *
 * 本模块只做计算，返回值就是下一个状态。
 * 状态或事件超出枚举范围时返回 SM_STATE_FAULT。
 * ============================================================================ */
#ifndef STATE_MACHINE_H
#define STATE_MACHINE_H

#include <stdint.h>

/* 状态枚举。SM_STATE_COUNT 不是真实状态，只表示状态个数（= 4）。 */
typedef enum {
    SM_STATE_IDLE = 0,    /* 空闲 */
    SM_STATE_ARMED,       /* 已使能 */
    SM_STATE_RUN,         /* 运行中 */
    SM_STATE_FAULT,       /* 故障 */
    SM_STATE_COUNT        /* 状态个数，不是真实状态 */
```

```c
} sm_state_t;

/* 事件枚举。SM_EVENT_COUNT 不是真实事件，只表示事件个数（= 6）。 */
typedef enum {
    SM_EVENT_NONE = 0,    /* 无事件 */
    SM_EVENT_ARM,         /* 使能 */
    SM_EVENT_START,       /* 启动 */
    SM_EVENT_STOP,        /* 停止 */
    SM_EVENT_FAULT,       /* 报故障 */
    SM_EVENT_CLEAR,       /* 清除故障 */
    SM_EVENT_COUNT        /* 事件个数，不是真实事件 */
} sm_event_t;

/* 用 switch (cur) 计算下一个状态并返回。
 *   cur 或 event 超出枚举范围 -> SM_STATE_FAULT
 *   未列出的组合               -> cur（保持原状态） */
sm_state_t sm_step(sm_state_t cur, sm_event_t event);

/* ---- 以下为 ★ 选做 ---- */

/* 状态表的一条规则：在 from 状态下收到 event，转到 to */
typedef struct {
    sm_state_t from;    /* 当前状态 */
    sm_event_t event;   /* 事件 */
    sm_state_t to;      /* 下一个状态 */
} sm_transition_t;

/* 在表里顺序查找第一条匹配 from + event 的规则，返回它的 to。
 *   table == NULL   -> cur（保持原状态）
 *   cur 或 event 超出枚举范围 -> SM_STATE_FAULT
 *   无匹配规则      -> cur（保持原状态）
 * table_len 是表项个数。 */
sm_state_t sm_step_table(sm_state_t cur, sm_event_t event,
                         const sm_transition_t *table, uint32_t table_len);

/* 用二维迁移表 SM_TABLE[cur][event] 查下一个状态并返回。
 * cur 或 event 超出枚举范围时返回 SM_STATE_FAULT。 */
sm_state_t sm_step_matrix(sm_state_t cur, sm_event_t event);

#endif /* STATE_MACHINE_H */
```

实现要求

1. `sm_step`：用 `switch (cur)`，每个 `case` 结尾 `break`； 先写 `sm_state_t nx = cur;`，各 `case` 只改 `nx`； 函数开头判 `cur` / `event` 是否超出枚举范围，超出就返回 `SM_STATE_FAULT`。
2. ★ `sm_step_table`：遍历结构体数组，第一条 `from == cur && event == event` 的表项决定返回值。
3. ★ `sm_step_matrix`：定义 `static const sm_state_t SM_TABLE[SM_STATE_COUNT][SM_EVENT_COUNT]`， 行是状态、列是事件；查表前先判下标范围。

验收用例

当前状态	事件	期望返回值
IDLE	NONE	IDLE
IDLE	ARM	ARMED
IDLE	START	IDLE
IDLE	FAULT	FAULT
ARMED	START	RUN
ARMED	ARM	ARMED
ARMED	STOP	IDLE
RUN	STOP	IDLE
RUN	START	RUN
FAULT	CLEAR	IDLE
FAULT	ARM	FAULT
`(sm_state_t)99`	ARM	`SM_STATE_FAULT`
IDLE	`(sm_event_t)99`	`SM_STATE_FAULT`

★ 选做：

场景	期望返回值
`sm_step_table(SM_STATE_IDLE, SM_EVENT_ARM, table, 8)`	`SM_STATE_ARMED`
`sm_step_table(SM_STATE_RUN, SM_EVENT_STOP, table, 8)`	`SM_STATE_IDLE`
`sm_step_table(SM_STATE_FAULT, SM_EVENT_ARM, table, 8)`	`SM_STATE_FAULT`（表里无此规则，保持原状态）
`sm_step_table(SM_STATE_IDLE, SM_EVENT_ARM, NULL, 8)`	`SM_STATE_IDLE`
`sm_step_matrix(SM_STATE_IDLE, SM_EVENT_ARM)`	`SM_STATE_ARMED`
`sm_step_matrix(SM_STATE_FAULT, SM_EVENT_CLEAR)`	`SM_STATE_IDLE`
`sm_step_matrix(SM_STATE_IDLE, (sm_event_t)99)`	`SM_STATE_FAULT`
`sm_step` / `sm_step_table` / `sm_step_matrix` 在全部 4 × 6 = 24 种组合下结果一致	三者返回值相同